

ДОКУМЕНТАЦИЯ ПРОДУКТА "NODE2CODE"

Апрель 2026

Содержание

1. О ПРОДУКТЕ	3
1.1. Что это такое.....	3
1.2. Для кого это	3
1.3. Ключевые особенности	4
2. СОСТАВ ПРОДУКТА	5
2.1. Плагин для Unity	5
2.2. Сайт	5
2.3. Серверная часть	6
3. ВОЗМОЖНОСТИ ПЛАГИНА.....	7
3.1. Типы нод.....	7
3.2. Как работает парсер (превращение кода в ноды)	9
3.3. Как работает генератор (превращение нод в код).....	10
3.4. Как работает выполнение графов.....	11
3.5. Работа с файлами	11
4. УСТАНОВКА ПЛАГИНА	12
5. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	13
5.1. Создание первого графа	13
5.2. Парсинг кода в граф.....	13
6. ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ	14
7. ЗАКЛЮЧЕНИЕ.....	15

1. О ПРОДУКТЕ

1.1. Что это такое

Node2Code - это инструмент для Unity, который позволяет создавать игровую логику двумя способами: через визуальные ноды (блок-схемы) и через написание кода на C#. Главная особенность в том, что эти два способа можно смешивать и конвертировать одно в другое.

Пользователь может:

- Нарисовать логику в виде нод и получить готовый C# скрипт
- Написать код на C# и увидеть его в виде понятной блок-схемы
- Выполнять созданную логику прямо в игре
- Сохранять и загружать графы в .cs файлы

Продукт создан для тех, кто только начинает изучать Unity и C#, а также для опытных разработчиков, которые хотят быстро прототипировать механики.

1.2. Для кого это

- Начинающие разработчики игр, которые хотят понять, как работает код
- Геймдизайнеры, которым нужно быстро проверить игровую механику
- Студенты и преподаватели игровых направлений
- Инди-разработчики, совмещающие визуальную логику и код

1.3. Ключевые особенности

Низкий порог входа. Для работы с Node2Code не нужно знать сложный синтаксис C#. Достаточно понимать базовые вещи: переменные, условия, циклы, действия.

Наглядность. Сложная логика, написанная в коде, превращается в понятную схему. Это помогает разобраться в чужом коде или вспомнить свою собственную логику через полгода.

Двусторонность. Можно начать с визуальных блоков, а когда станет тесно - перейти к коду. И наоборот: написать сложный алгоритм в коде, а потом вывести его в ноды для презентации или обучения.

Работа в реальном времени. Созданную логику можно сразу запустить в игре, не выходя из редактора.

Сохранение. Графы сохраняются в .cs, их удобно передавать коллегам, хранить в git и версионировать.

2. СОСТАВ ПРОДУКТА

Продукт состоит из трёх частей:

2.1. Плагин для Unity

Это основная часть. Плагин встраивается в Unity и добавляет в меню Tools пункт для открытия редактора нод.

Что внутри:

- Редактор, где можно рисовать графы из нод
- Более 30 типов нод: переменные, сравнения, условия, циклы, методы, классы
- Парсер, который превращает C# код в граф
- Генератор, который превращает граф в C# код
- Система выполнения, которая запускает граф прямо в игре

2.2. Сайт

Сайт, где пользователь может узнать о продукте и скачать его.

Что внутри:

- Описание возможностей плагина
- Инструкция по работе и установке плагина
- Личный кабинет
- Тёмная и светлая тема оформления

2.3. Серверная часть

Бэкенд, который хранит данные пользователей.

Что внутри:

- Регистрация и вход пользователей
- Хранение прогресса инструкции
- API для связи сайта с базой данных

3. ВОЗМОЖНОСТИ ПЛАГИНА

3.1. Типы нод

Все ноды разделены на группы:

Литералы:

- Целые числа (int)
- Дробные числа (float)
- Правда/Ложь (bool)
- Текст (string)

Математика:

- Сложение (+)
- Вычитание (-)
- Умножение (*)
- Деление (/)
- Остаток от деления (%)
- Модуль
- Max
- Min

Сравнения:

- Равно (==)
- Не равно (!=)
- Больше (>)
- Меньше (<)
- Больше или равно (>=)
- Меньше или равно (<=)

Логика:

- И (&&)
- ИЛИ (||)
- НЕ (!)

Управление потоком:

- If (условие)
- Else (иначе)
- For (цикл со счётчиком)
- While (цикл с условием)
- Debug.Log (вывод в консоль)

Конвертация:

- int.Parse - текст в число
- float.Parse - текст в дробное число
- ToString - что угодно в текст

А также ноды методов и классов, которые создаёт пользователь

3.2. Как работает парсер (превращение кода в ноды)

Парсер читает C# код и превращает его в граф.

Что он понимает:

- Объявление переменных: `int x = 10;`
- Присваивание: `x = 20;`
- Составные операции: `x += 5;`
- Инкремент и декремент: `x++; ++x; x--; --x;`
- Арифметику: `+` `-` `*` `/` `%`
- Сравнения: `==` `!=` `>` `<` `>=` `<=`
- Логику: `&&` `||` `!`
- Условия: `if` / `else`
- Циклы: `for`, `while`
- Вывод в консоль: `Degub.Log()`
- Преобразование типов: `int.Parse()`, `float.Parse()`, `ToString()`
- Методы и классы, созданные пользователем

Парсер специально сделан так, чтобы пользователь мог писать код без лишних сложностей. Не нужно объявлять класс и метод - код оборачивается автоматически. Можно просто писать инструкции подряд, как в школьной тетради.

Что парсер НЕ понимает (пока что):

- Массивы и списки
- Сложные классы и структуры
- События и делегаты
- LINQ запросы

- Switch оператор
- Do-while цикл
- Foreach цикл

3.3. Как работает генератор (превращение нод в код)

Генератор обходит граф, начиная с корневых нод (тех, у которых нет входа по потоку выполнения), и последовательно генерирует C# код.

Он умеет:

- Объявлять переменные с правильными типами
- Генерировать выражения любой вложенности
- Создавать условия if/else с правильными блоками
- Генерировать циклы for и while
- Обрабатывать присваивания и операции
- Сохранять исходные выражения через ExpressionOverride (чтобы при обратном парсинге не потерялась красота кода)

3.4. Как работает выполнение графов

Граф можно выполнить прямо в Unity, без генерации кода.

Система выполнения:

1) Автоматическая:

- Определяет стартовые ноды
- Выполняет ноды в правильном порядке
- Передаёт значения по связям между нодами
- Обрабатывает ветвления (true/false у условий)
- Поддерживает вложенные графы (для условий и тел циклов)
- Хранит переменные в контексте выполнения

2) Ручная:

- Определяет последовательность через связь построенную между нодами за счёт портов `exesin` и `exesout`

3.5. Работа с файлами

Пользователь может:

- Создать новый граф
- Сохранить в `.cs` файл
- Загрузить код из `.cs` файла

4. УСТАНОВКА ПЛАГИНА

1. Скачайте файл Node2Code.unitypackage с сайта
2. Откройте ваш проект в Unity
3. Нажмите Assets -> Import Package -> Custom Package
4. Выберите скачанный файл
5. Нажмите Import
6. Откройте: Window → Package Manager
7. Нажмите кнопку + (плюс) в левом верхнем углу
8. Выберите Add package from git URL...
9. Вставьте ссылку:
<https://github.com/Warwlock/NodeGraphProcessor.git>
10. Нажмите Add
11. В Package Manager снова нажмите +
12. Выберите Add package by name...
13. Введите:
com.unity.nuget.newtonsoft-json
14. Нажмите Add
15. Откройте плагин: в меню Unity выберите Tools → Node2Code

5. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1. Создание первого графа

1. Откройте Tools -> Node2Code
2. Нажмите "Создать новый граф"
3. Через ПКМ -> Create Node -> «Тип ноды» -> «Нода» создайте ноды
4. Соедините порты нод, перетаскивая мышью от одного порта к другому
5. Нажмите "Сгенерировать код" - получите C# скрипт

5.2. Парсинг кода в граф

1. Откройте Tools -> Node2Code
2. В левой панели введите код C#
3. Нажмите "Парсить код"
4. Справа появится визуальный граф, соответствующий коду

6. ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ

- Unity 6 (6000.0) LTS или новее
- Windows, Mac или Linux

7. ЗАКЛЮЧЕНИЕ

Node2Code - это инструмент, который делает обучение Unity и C# наглядным и понятным. Он подходит для полных новичков, которые боятся кода, и для опытных разработчиков, которые хотят быстро прототипировать механики.

Двусторонняя конвертация между нодами и кодом позволяет выбрать тот способ работы, который удобен в данный момент. Можно начать с визуальных блоков, а когда станет тесно - перейти к коду. И наоборот: написать сложный алгоритм в коде, а потом показать его в виде понятной схемы.

Продукт создан командой из пяти человек, каждый из которых внёс свой вклад в разработку, дизайн, тестирование и документацию.